

모두를 위한 딥러닝 (입문편)

허 찬

강의 계획표 - 모두를 위한 딥러닝 (입문편)

학습주제(강좌명)	차시(모듈)	차시별 주제	학습내용
모두를 위한 딥러닝 (입문편)	1	Dive into deep learning	머신러닝, 딥러닝, 인공신경망의 개념 이해 딥러닝의 분류 및 응용 분야 이해 딥러닝 모델의 장/단점 이해
	2	딥러닝 코딩 환경 준비	머신러닝/딥러닝 워크플로우의 이해 파이썬 기반 머신러닝, 딥러닝 프레임워크의 이해 패키지 설치 및 기초 벡터 연산 설명
	3	딥러닝 모델 학습의 4steps	딥러닝 학습 과정의 단계별 이해
	4	데이터 시각화	데이터 시각화 Pandas, Matplotlib, Seaborn 패키지의 사용
	5	딥러닝을 이용한 분류 모델 실습	딥러닝 코드 실습을 통한 분류 문제 해결
	6	딥러닝을 이용한 회귀 모델 실습	딥러닝 코드 실습을 통한 회귀 문제 해결
	7	딥러닝 모델의 성능 향상 기법	딥러닝 기본 모델의 성능 향상 기법

6차시

1.6	딥러닝을 이용한 회귀 모델 실습	딥러닝 코드 실습을 통한 회귀 문제 해결
-----	-------------------	------------------------

6차시 수업

- 딥러닝을 이용한 문제 해결 과정
- 딥러닝 코드 실습을 통한 회귀 문제 해결
- 다양한 데이터 전처리 과정 실습

1.6 딥러닝을 이용한 회귀 문제 실습

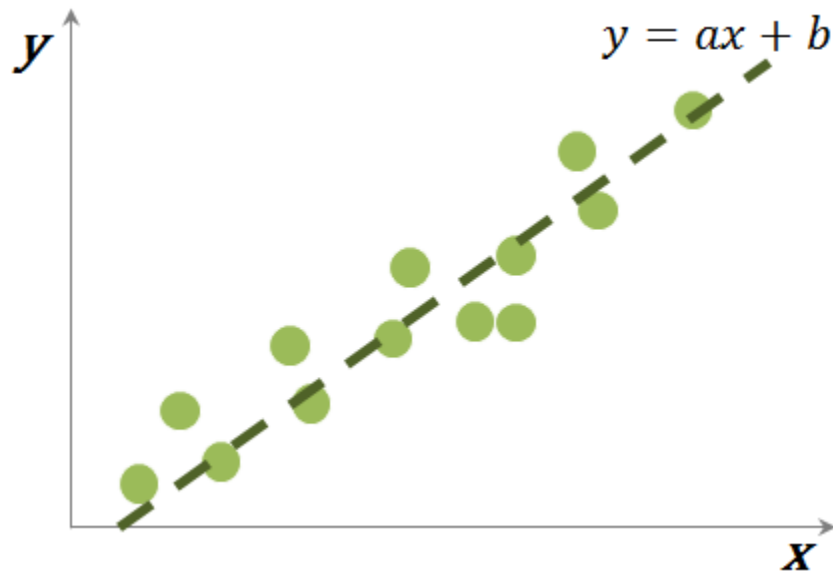
딥러닝을 이용한 문제 해결 과정

- 0. 문제 정의
- 1. 데이터 정의
- 2. 모델 정의
- 3. 모델 구현 및 학습
- 4. 모델 테스트 및 시각화

1.6 딥러닝을 이용한 회귀 문제 실습

회귀 문제란?

연속적인 값을 예측하는 문제!



1.6 딥러닝을 이용한 회귀 문제 실습



Regression



What will be the temperature tomorrow?



Classification



Will it be hot or cold tomorrow?

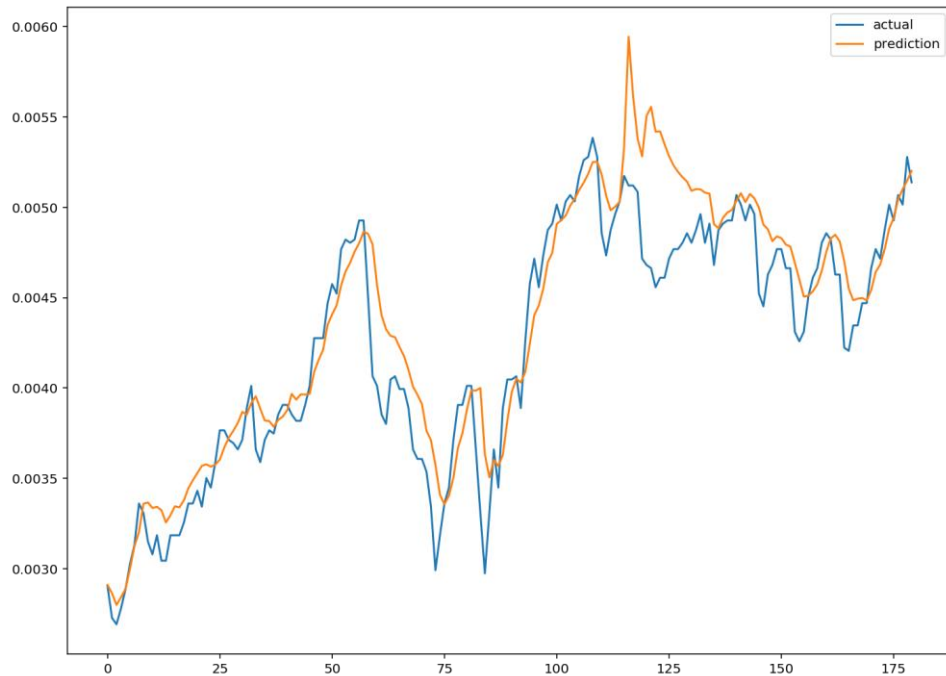


- 명확한 정답 수치를 내는 회귀 문제

1.6 회귀 문제 응용분야 예시

1. 금융 분야 - 주가 예측:

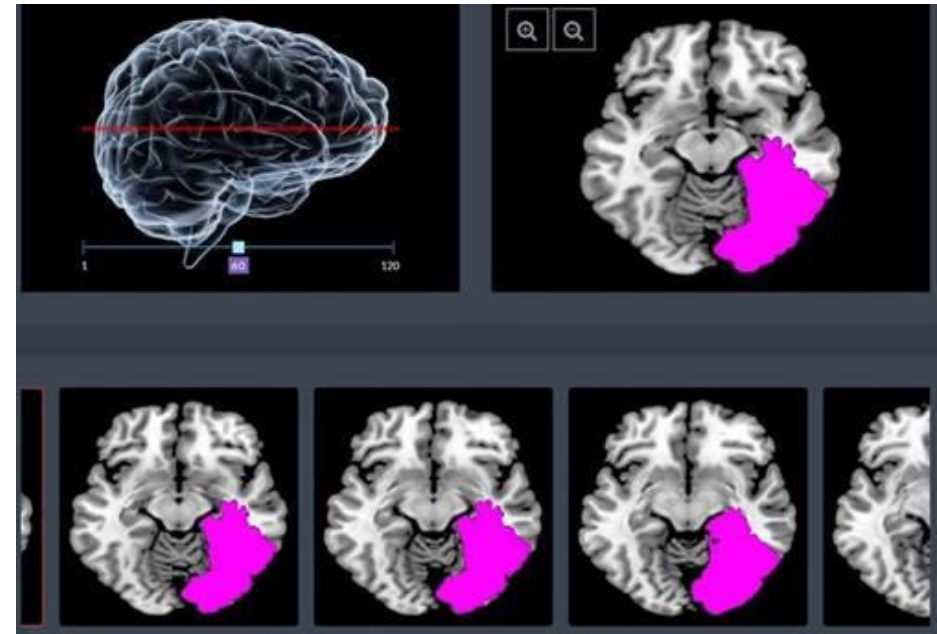
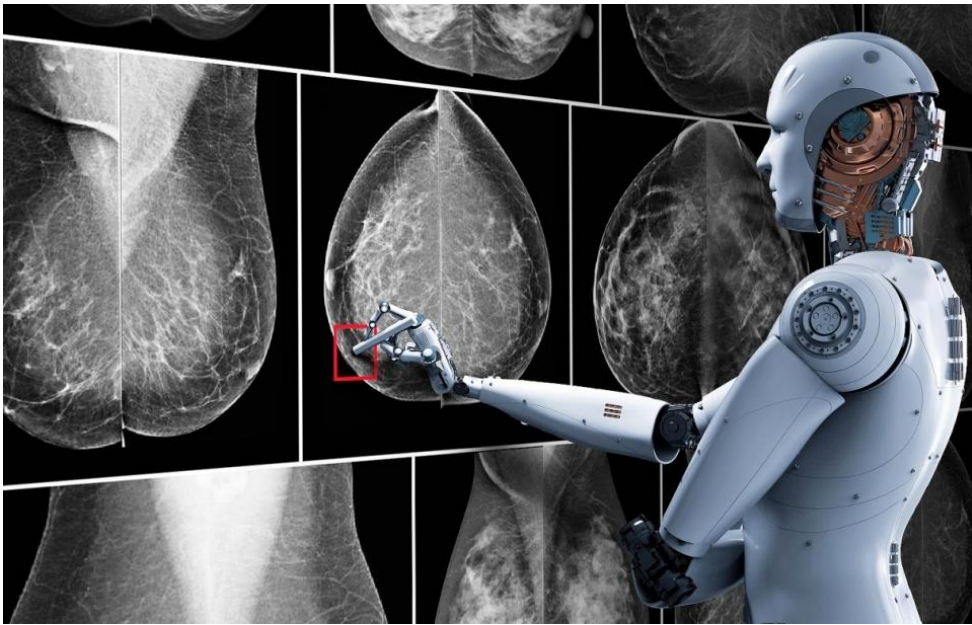
- 금융 시장에서는 딥러닝을 사용하여 주가 예측 모델을 개발하고 있습니다. 이 모델은 과거의 주가 데이터, 시장 지표 및 기타 경제적 요인을 입력으로 받아 주가의 추세를 예측합니다. 이를 통해 투자자들은 미래의 주가 움직임에 대한 통찰력을 얻을 수 있습니다.



1.6 회귀 문제 응용분야 예시

2. 의료 분야 - 환자 건강 상태 예측:

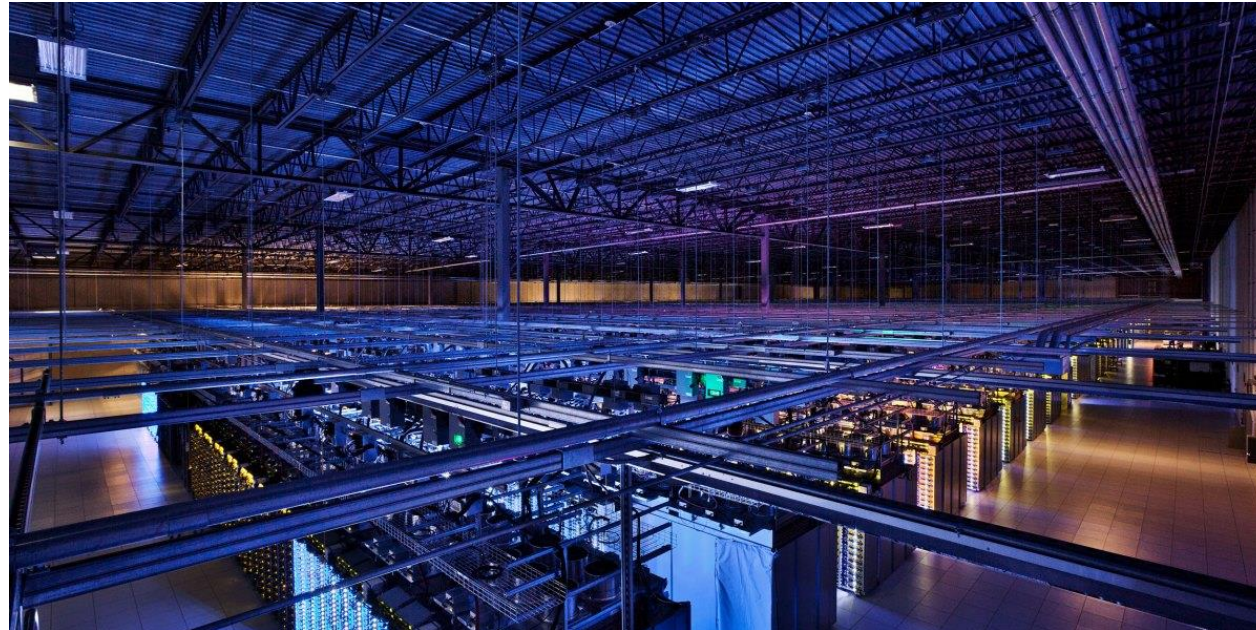
- 의료 분야에서는 딥러닝을 사용하여 환자의 건강 상태를 예측하는데 활용하고 있습니다. 환자의 의료 기록, 생체 신호 및 기타 관련 정보를 모델에 입력하여 향후 질병 발생 또는 건강 상태의 변화를 예측합니다. 이는 조기 진단 및 개별화된 치료 계획을 수립하는 데 도움이 됩니다.



1.6 회귀 문제 응용분야 예시

3. 에너지 분야 - 전력 소비 예측:

- 에너지 기업은 딥러닝을 사용하여 전력 소비를 예측하여 효율적으로 에너지를 관리하고 비용을 절감합니다. 또한 에너지 절약을 위한 최적의 상황을 딥러닝을 이용하여 모델링 할 수 있습니다.



- 데이터센터 냉방에 필요한 에너지를 40% 절감 (Google)

1.6 회귀 문제 응용분야 예시

4. 제조 분야 - 제품 품질 예측:

- 제조업에서는 딥러닝을 사용하여 제품의 품질을 예측하고 불량품을 사전에 감지하는데 활용합니다. 센서 데이터, 제조 공정 기록 및 이력 등을 모델에 입력하여 제품 불량률을 예측하고 생산 프로세스를 최적화합니다.

5. 소매 분야 - 수요 예측:

소매 업계에서는 딥러닝을 사용하여 제품의 수요를 예측하여 재고를 최적화하고 생산 계획을 조정합니다. 소비자 행동, 마케팅 캠페인, 경쟁사 정보 등을 고려하여 향후 수요를 예측하고 재고 관리에 활용합니다.

⋮

1.6 딥러닝을 이용한 회귀 문제 실습

딥러닝을 이용한 회귀 문제 실습

1. 딥러닝을 이용한 집값 데이터 예측 모델 (csv 데이터)
 - 회귀 문제를 위한 딥러닝 모델 설계 방법
 - 회귀 문제를 위한 loss function
2. 딥러닝을 이용한 자동차 연비 예측
3. 딥러닝을 이용한 자전거 대여량 예측
4. 딥러닝을 이용한 냉/난방 효율 예측

1.6 딥러닝을 이용한 부동산 가격 예측

Point!

- 기본적인 회귀 모델 설계 방법
- 회귀모델을 위한 loss function

1.6 딥러닝을 이용한 부동산 가격 예측

문제 정의:

- 부동산과 관련된 여러 데이터를 가지고 부동산의 가격을 예측하는 회귀 모델 설계

데이터 정의

특징 변수:

- CRIM: 인구 1인당 범죄율
- ZN: 25,000 평방 피트 이상의 주거 구역 비중
- INDUS: 소매업 외 상업이 차지하는 면적 비율
- CHAS: 찰스 강(Charles River)에 대한 더미 변수 (강 주변이면 1, 아니면 0)
- NOX: 일산화질소 농도
- RM: 주택당 평균 방 수
- AGE: 1940년 이전에 건축된 주택의 비율
- DIS: 5개의 보스턴 고용 센터까지의 가중 거리
- RAD: 순환고속도로 접근성 지수
- TAX: \$10,000당 부동산 세율
- PTRATIO: 학생-교사 비율
- B: 흑인 비율
- LSTAT: 저소득층의 비율



타겟 변수 (종속 변수):

- MEDV: 주택의 중간 가격 (1,000 달러 단위)

1.6 딥러닝을 이용한 부동산 가격 예측

1. Module import & data preprocessing

```
from keras.datasets import boston_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 보스턴 주택 가격 데이터셋 로드
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()

# 데이터 전처리: 표준화 (Standardization)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 데이터 분할: 훈련 데이터와 검증 데이터
X_train_split, X_val_split, y_train_split, y_val_split = train_test_split(X_train_scaled,
y_train, test_size=0.2, random_state=42)
```

1.6 딥러닝을 이용한 부동산 가격 예측

2. Model build

```
from keras.models import Sequential
from keras.layers import Dense

# 모델 정의
model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(units=1, activation='linear'))

# 모델 컴파일
model.compile(optimizer='adam', loss='mean_squared_error')

# 모델 학습
history = model.fit(X_train_split, y_train_split, epochs=100,
                    batch_size=32, validation_data=(X_val_split, y_val_split), verbose=0)
```

1.6 딥러닝을 이용한 부동산 가격 예측

3. Model test

모델 평가

```
mse_train = model.evaluate(X_train_scaled, y_train, verbose=0)
mse_test = model.evaluate(X_test_scaled, y_test, verbose=0)
print(f'Training Mean Squared Error: {mse_train}')
print(f'Test Mean Squared Error: {mse_test}')
```

새로운 데이터에 대한 예측

```
new_data = X_test_scaled[:5] # 예제로 처음 5개의 데이터 사용
predictions = model.predict(new_data)

print("\nPredictions:")
for i in range(5):
    print(f"Prediction: {predictions[i][0]:.2f}, Actual: {y_test[i]:.2f}")
```

Mean square error

$$MSE = \frac{1}{n} \sum \left(\underbrace{y - \hat{y}}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}} \right)^2$$

- 모델 마지막 node의 output(예측값): $\hat{y}$
- Label (정답) : y
- 예측값과 정답의 차이가 작을수록 손실 줄어듦

1.6 딥러닝을 이용한 자동차 연비 예측

Point!

- CSV 데이터의 기초 처리 방법
- (Option) 회귀모델 데이터의 시각화 과정
- 범주형 데이터의 전처리
- 신경망을 이용한 회귀 모델링

1.6 딥러닝을 이용한 자동차 연비 예측



- 차량 구매시 연비는 중요 사항! 그런데 연비가 진짜일까...?

1.6 딥러닝을 이용한 자동차 연비 예측

문제 정의:

- 자동차의 여러 데이터를 가지고 연비를 예측하는 회귀 모델 설계

데이터 정의:

특징 변수:

- Cylinders: 엔진 실린더의 수를 나타내는 특징으로, 자동차의 성능과 효율성에 영향을 미칩니다.
- Displacement: 엔진의 배기량을 나타내는 특징으로, 자동차의 크기와 연비에 영향을 미칩니다.
- Horsepower: 자동차의 마력을 나타내는 특징으로, 성능과 연비에 영향을 미칩니다.
- Weight: 자동차의 무게를 나타내는 특징으로, 연비에 영향을 미치는 중요한 요소 중 하나입니다.
- Acceleration: 0에서 60 mph까지 가속하는데 걸리는 시간을 나타내는 특징으로, 성능과 연비에 영향을 미칩니다.
- Model Year: 자동차 모델의 제조 연도를 나타내는 특징으로, 연식에 따라 기술적인 특성이 변할 수 있습니다.
- Origin: 자동차의 제조 국가를 나타내는 특징으로, 범주형 데이터입니다. 이 데이터셋에서는 1, 2, 3으로 표현되며 각각 미국, 유럽, 일본을 나타냅니다.

타겟 변수:

- MPG (Miles Per Gallon): 주행거리 당 연료 소비량을 나타내는 특징으로, 자동차의 연비를 나타냅니다.

1.6 딥러닝을 이용한 자동차 연비 예측

1. Module import & data load

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from keras.models import Sequential
from keras.layers import Dense

# 데이터 로드
url = "http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower', 'Weight', 'Acceleration', 'Model Year',
                'Origin']
raw_data = pd.read_csv(url, delim_whitespace=True, names=column_names, na_values='?', comment='\t',
                       skipinitialspace=True)
```

1.6 딥러닝을 이용한 자동차 연비 예측

2. Data preprocess

```
# 'Origin' 특징에 대한 원-핫 인코딩 적용
data = pd.get_dummies(raw_data, columns=['Origin'], prefix='Origin')

# 누락된 데이터 제거
data = data.dropna()

# 특징과 레이블 분리
X = data.drop('MPG', axis=1)
y = data['MPG']

# 데이터 전처리: 표준화 (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 데이터 분할: 훈련 데이터와 검증 데이터
X_train, X_val, y_train, y_val = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

- Origin 특징에 대한 원-핫 인코딩?

1.6 딥러닝을 이용한 자동차 연비 예측

(Option) 데이터 시각화

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터 로드
url = "http://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
column_names = ['MPG', 'Cylinders', 'Displacement', 'Horsepower', 'Weight', 'Acceleration', 'Model Year',
                'Origin']
raw_data = pd.read_csv(url, delim_whitespace=True, names=column_names, na_values='?', comment='\t',
                       skipinitialspace=True)

# 'Origin' 특징에 대한 원-핫 인코딩 적용
data = pd.get_dummies(raw_data, columns=['Origin'], prefix='Origin')

# 누락된 데이터 제거
data = data.dropna()

# 기술 통계 정보 확인
print(data.describe())
```

1.6 딥러닝을 이용한 자동차 연비 예측

(Option) 데이터 시각화

히스토그램 시각화

```
data.hist(bins=20, figsize=(15, 10))  
plt.show()
```

상관 행렬 시각화

```
correlation_matrix = data.corr()  
plt.figure(figsize=(10, 8))  
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")  
plt.title("Correlation Matrix")  
plt.show()
```

Pairplot 시각화

```
sns.pairplot(data, diag_kind='kde')  
plt.show()
```

'Origin'에 따른 연비 Boxplot

```
plt.figure(figsize=(10, 6))  
sns.boxplot(x='Origin', y='MPG', data=data)  
plt.title("Boxplot of MPG by Origin")  
plt.show()
```

1.6 딥러닝을 이용한 자동차 연비 예측

3. Model 구성 및 테스트

신경망 모델

```
model = Sequential()  
model.add(Dense(units=64, activation='relu', input_dim=X_train.shape[1]))  
model.add(Dense(units=1, activation='linear'))  
model.compile(optimizer='adam', loss='mean_squared_error')
```

모델 학습

```
model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=0)
```

예측 및 평가

```
y_pred_nn = model.predict(X_val)  
mse_nn = mean_squared_error(y_val, y_pred_nn)  
print(f'Mean Squared Error (Neural Network): {mse_nn:.2f}')
```

시각화: 실제 vs. 예측값

```
plt.subplot(1, 2, 2)  
sns.scatterplot(x=y_val, y=y_pred_nn.flatten())  
plt.title('Neural Network: Actual vs. Predicted')  
plt.show()
```

범주형 데이터의 처리

Acceleration	Model Year	Origin	MPG
12.0	70	U.S.	18.0
11.5	70	U.S.	15.0
11.0	70	U.S.	18.0
12.0	70	U.S.	16.0
10.5	70	U.S.	17.0

Acceleration	Model Year	Origin_US	Origin_Europe	Origin_Japan	MPG
12.0	70	1	0	0	18.0
11.5	70	1	0	0	15.0
11.0	70	1	0	0	18.0
12.0	70	1	0	0	16.0
10.5	70	1	0	0	17.0

- 범주형 데이터(Origin)의 전처리 이전(위)/ 전처리 이후(아래)

범주형 데이터의 처리???

범주형 데이터의 처리 이유

1. 모델의 이해성을 높이기

원-핫 인코딩은 범주형 데이터를 이진 형태로 표현하는 방법 중 하나. 이는 모델이 범주 간의 순서나 관계를 학습하지 않도록 하며, 각 범주를 독립적인 개체로 취급. 이로써 모델이 범주 간의 관계를 잘못 이해하거나 오해하는 경우를 방지할 수 있음.

2. 범주형 데이터의 순서를 배제

원-핫 인코딩을 사용하면 범주형 데이터의 순서가 사라지기 때문에 모델이 범주 간의 상대적인 순서를 고려하지 않습니다. 예를 들어, "국가" 특징이 있을 때, 미국(1), 유럽(2), 일본(3)과 같이 숫자로 표현하면 모델은 이 숫자의 크기를 이용하여 상대적인 크기나 순서를 가정할 수 있음. 그러나 원-핫 인코딩을 사용하면 각 국가는 독립적인 특성으로 표현되어 순서가 없어짐.

3. 다양한 범주형 변수 다루기

원-핫 인코딩은 다양한 범주형 변수에 유연하게 대응 가능. 범주의 수가 많아지더라도 각 범주에 대한 이진 특성만 추가하면 되므로 모델의 복잡성을 증가시키지 않으면서도 다양한 범주형 변수를 효과적으로 표현할 수 있음.

예시

만약 "국가"가 미국, 유럽, 일본으로 표현된 범주형 변수라고 가정하면:

•숫자로 표현한 경우:

- 미국: 1
- 유럽: 2
- 일본: 3

•원-핫 인코딩으로 표현한 경우:

- 미국: 1, 0, 0
- 유럽: 0, 1, 0
- 일본: 0, 0, 1

이렇게 하면 각 국가는 독립적인 특성으로 표현되어 모델이 순서를 학습하지 않고 범주 간의 영향을 독립적으로 고려할 수 있음.

1.6 딥러닝을 이용한 자전거 대여량 예측

Point!

- 범주형 데이터와 수치형 데이터가 섞여 있는 데이터의 전처리
- 신경망을 이용한 회귀 모델링
- 모델 저장 및 로드

1.6 딥러닝을 이용한 자전거 대여량 예측

데이터 정의

- "Bike Sharing Demand" 데이터셋은 Capital Bikeshare에서 제공한 워싱턴 D.C.의 자전거 대여 데이터. 이 데이터셋은 자전거 대여소에서 대여된 자전거의 일일 대여 횟수를 기록한 것이며, 다양한 환경 변수들이 포함되어 있습니다.
- 데이터셋의 주요 특징 및 타겟 변수에 대한 설명은 다음과 같습니다:

특징 변수 (독립 변수):

- season: 계절 (1: 봄, 2: 여름, 3: 가을, 4: 겨울)
- holiday: 공휴일 여부 (1: 공휴일, 0: 비공휴일)
- workingday: 근무일 여부 (1: 근무일, 0: 휴무일)
- weather: 날씨 상태 (1: 맑음, 2: 흐림, 3: 가벼운 비/눈, 4: 심한 비/눈)
- temp: 온도 (섭씨)
- atemp: 체감 온도 (섭씨)
- humidity: 상대 습도
- windspeed: 풍속

타겟 변수 (종속 변수):

- count: 대여 횟수



1.6 딥러닝을 이용한 회귀 문제 실습

1. Module import 및 데이터 로드

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from keras.models import Sequential, load_model
from keras.layers import Dense

# Bike Sharing Demand 데이터셋 로드 (train.csv 파일 사용)
csv_path = "bike_train.csv"
data = pd.read_csv(csv_path)
```

1.6 딥러닝을 이용한 회귀 문제 실습

2. 데이터 전처리

필요한 특징과 타겟 변수 선택

```
features = ['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp',  
            'humidity', 'windspeed']  
target = 'count'
```

특징과 타겟 변수 추출

```
X = data[features]  
y = data[target]
```

범주형 데이터에 대한 원-핫 인코딩

```
X_categorical = pd.get_dummies(X, columns=['season', 'holiday', 'workingday',  
            'weather'], drop_first=True)
```

수치형 데이터 추출

```
X_numeric = X[['temp', 'atemp', 'humidity', 'windspeed']]
```

데이터 합치기

```
X_processed = pd.concat([X_categorical, X_numeric], axis=1)
```

1.6 딥러닝을 이용한 회귀 문제 실습

2. 데이터 전처리

```
# 데이터 전처리: 표준화 (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_processed)

# 데이터 분할: 훈련 데이터와 검증 데이터
X_train, X_val, y_train, y_val = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
```

1.6 딥러닝을 이용한 회귀 문제 실습

3. 모델 정의 및 학습

모델 정의

```
model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(units=1, activation='linear'))
model.compile(optimizer='adam', loss='mean_squared_error')
```

모델 훈련

```
model.fit(X_train, y_train, epochs=100, batch_size=32, validation_data=(X_val, y_val), verbose=2)
```

모델 저장

```
model.save("bike_sharing_model.h5")
print("모델이 저장되었습니다.")
```

저장한 모델 불러오기

```
loaded_model = load_model("bike_sharing_model.h5")
print("모델이 불러와졌습니다.")
```

저장한 모델로 예측 수행

```
y_pred = loaded_model.predict(X_val)
```

예측 결과 출력

```
print("\n대여 횟수 예측 결과:")
print("실제 값\t\t\t\t\t예측 값")
for true_val, pred_val in zip(y_val[:5], y_pred[:5]):
    print(f"{true_val}\t\t\t\t{pred_val[0]:.4f}")
```

.h5 형태로 모델 저장

1.6 딥러닝을 이용한 냉난방 효율 예측

Point!

- xlsx 데이터의 기초 처리 방법
- Colab에서 data 업로드 및 사용 방법
- Input data가 두가지 이상을 예측해야 하는 경우

1.6 딥러닝을 이용한 냉난방 효율 예측

문제 정의:

- 대기중

데이터셋의 특징(독립 변수)은 다음과 같습니다:
(데이터셋 내부에서는 X1~X8로 표기)

- Relative Compactness: 상대적인 조밀도
- Surface Area: 표면적
- Wall Area: 벽면적
- Roof Area: 지붕면적
- Overall Height: 전체 높이
- Orientation: 건물의 방향
- Glazing Area: 창문면적
- Glazing Area Distribution: 창문면적 분포

타겟 변수(종속 변수)은 다음과 같이 두 가지입니다:
(데이터셋 내부에서는 Y1,Y2로 표기)

- Heating Load: 난방 에너지 효율성
- Cooling Load: 냉방 에너지 효율성



1.6 딥러닝을 이용한 냉난방 효율 예측

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from keras.models import Sequential
from keras.layers import Dense
```

```
# Energy Efficiency 데이터셋 로드
csv_path = "ENB2012_data.xlsx"
data = pd.read_excel(csv_path)
```

```
# 실제 데이터 컬럼 확인
print(data.columns)
```

```
# 타겟 데이터 추출 (난방 및 냉방 에너지 효율성)
y_heating = data['Y1']
y_cooling = data['Y2']
```

```
# 특징 데이터 추출
features = data.drop(['Y1', 'Y2'], axis=1)
X = features
```

```
# 데이터 전처리: 표준화 (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# 데이터 분할: 훈련 데이터와 검증 데이터 ##Input은 1개, 예측해야할 것은 2개의 다른 시나리오##
X_train, X_val, y_train_heating, y_val_heating, y_train_cooling, y_val_cooling =
train_test_split(X_scaled, y_heating, y_cooling, test_size=0.2, random_state=42)
```

1. Module import & data load & data preprocessing

1.6 딥러닝을 이용한 냉난방 효율 예측

2. 모델 정의 및 학습

```
# 모델 정의 (난방 에너지 효율성 예측)
```

```
model_heating = Sequential()  
model_heating.add(Dense(units=64, activation='relu', input_dim=X_train.shape[1]))  
model_heating.add(Dense(units=1, activation='linear'))  
model_heating.compile(optimizer='adam', loss='mean_squared_error')
```

```
# 모델 정의 (냉방 에너지 효율성 예측)
```

```
model_cooling = Sequential()  
model_cooling.add(Dense(units=64, activation='relu', input_dim=X_train.shape[1]))  
model_cooling.add(Dense(units=1, activation='linear'))  
model_cooling.compile(optimizer='adam', loss='mean_squared_error')
```

```
# 각 모델에 대해 학습 수행
```

```
history_heating = model_heating.fit(X_train, y_train_heating, epochs=100, batch_size=32,  
                                   validation_data=(X_val, y_val_heating), verbose=2)
```

```
history_cooling = model_cooling.fit(X_train, y_train_cooling, epochs=100, batch_size=32,  
                                   validation_data=(X_val, y_val_cooling), verbose=2)
```

1.6 딥러닝을 이용한 냉난방 효율 예측

3. 모델 테스트

```
# 각 모델에 대해 예측 수행
y_pred_heating = model_heating.predict(X_val)
y_pred_cooling = model_cooling.predict(X_val)

# 예측 결과 출력
print("난방 에너지 효율성 예측 결과:")
print("실제 값\t\t\t\t\t예측 값")
for true_val, pred_val in zip(y_val_heating[:5], y_pred_heating[:5]):
    print(f"{true_val:.4f}\t\t\t\t{pred_val[0]:.4f}")

print("\n냉방 에너지 효율성 예측 결과:")
print("실제 값\t\t\t\t\t예측 값")
for true_val, pred_val in zip(y_val_cooling[:5], y_pred_cooling[:5]):
    print(f"{true_val:.4f}\t\t\t\t{pred_val[0]:.4f}")
```

감사합니다